Lodos v2: Installation Guide

From a fresh clone to a working installation: the simulator, the viewer, the notebooks, the tools and the documents

As of 2026-11-04. Every command was run on a fresh clone, on Ubuntu 20.04 with an RTX 4080 (16 GiB, driver 580.126.09) and uv 0.12.5, on 2026-10-27 or 2026-11-04; the text dates its measurements, and says which steps were not run on the code as of 2026-11-04. The other documents: *Development Guide* (`docs/development_guide.pdf`), *Reading Guide* (`docs/reading_guide.pdf`), *The Live Viewer* (`docs/visualizer.pdf`), the manuals *Collision Detection* (`docs/collision_detection.pdf`) and *Contact Dynamics* (`docs/contact_dynamics.pdf`), and the guides *Creating a Simulation from an MJCF* (`docs/creating_new_sim_from_mjcf.pdf`) and *Performance Tuning* (`docs/performance_tuning.pdf`). Built from `docs/manual/`.

About this document. You have a Linux machine with an NVIDIA GPU and the repository, or its URL, and want Lodos v2 to work: the simulator, the live viewer, the notebooks, the tuning tools, the mesh pipeline and the documents. This guide goes from the clone to all of that, one command at a time, and says what to check after each.

- what the machine needs (Section 2);
- the recommended installation with uv, and what it puts where (Sections 4 and 5);
- the pip alternatives, from a clone or from a git URL (Section 6);
- how to check the installation: the GPU, a first simulation, the notebooks and the viewer (Section 7);
- the optional tools: the tuning tools, the mesh pipeline's external programs and the documents' toolchain (Sections 8, 10 and 11);
- machines without a display (Section 12), updates (Section 13) and what goes wrong (Section 14).

Each step was run in a fresh clone, from a shell that inherited nothing: no activated environment and, on 2026-10-27, an empty uv cache, an empty Warp kernel cache and no Python 3.14 on the path. On 2026-11-04 the installation itself (the uv environment, the wheel, the three pip routes, the imports and the first simulation) was run again in a new clone, with uv's cache and Warp's kernel cache warm and each pip route from an empty pip cache of its own. The installation's sizes and the pip routes' times are from that run; uv's time with an empty cache, and what the steps not run again measured, are from the first. The downloads were on a fast network. Where a step was not run, or not run again, the text says so.

Contents

1 The short version 2

2 Prerequisites 3

2.1 The machine . . . 3

2.2 The driver . . . 4

2.3 Software . . . 4

3 Getting the code 4

4 The recommended installation: uv 5

4.1 One command . . . 5
4.2 The dependency groups . . . 5
4.3 Using the environment . . . 6

5 What is installed, and from where **6**
5.1 Assets and caches . . . 6

6 Installing with pip **6**
6.1 Editable, from a clone . . . 6
6.2 A regular install, from a clone . . . 7
6.3 From a git URL, without a clone . . . 7
6.4 What each route gives . . . 8

7 Verifying the installation **8**
7.1 Warp and the GPU . . . 8
7.2 A first simulation . . . 9
7.3 The notebooks . . . 9
7.4 The live viewer . . . 9

8 The tuning tools **10**

9 Newton's sims, for comparison **10**

10 The mesh pipeline's tools **11**
10.1 Instant Meshes, for steps 3 and 4 . . . 11
10.2 CoACD, manifold3d and scikit-image, for steps 1 and 2 . . . 11
10.3 PaMO, for step 2 . . . 12
10.4 CGAL and step 1: a Python 3.12 environment . . . 13
10.5 The windows . . . 13

11 The documents' toolchain **13**

12 Machines without a display **14**

13 Keeping an installation current **15**

14 Troubleshooting **15**

1 The short version

```
git clone https://github.com/martius-lab/lodos && cd lodos
uv sync --locked --all-groups                      # Python 3.14 and every dependency
.venv/bin/jupyter nbconvert --to notebook --execute --output-dir /tmp/nb \
    notebooks/01_quickstart.ipynb                  # the solver, both modes, on the GPU
.venv/bin/lodos-viewer arm_5_sdf                   # the live viewer (a display)
```

That is a working installation of everything except Newton's sims in the viewer, the mesh pipeline's external programs and the documents' toolchain. Table 1 lists what each part needs.

Table 1: What each part of the repository needs beyond a clone.

part	needs	where
the simulator (`lodos`), the built-in scenes and their assets (`viewer/scenes`, `viewer/assets`)	an NVIDIA GPU and driver; `uv sync`, or `pip install`	Sections 2, 4 and 6
the live viewer	the `vis` group or extra; an OpenGL display	Section 7.4 and Section 12
Newton's sims, in the viewer	uv; `viewer/newton/`'s own environment	Section 9
the notebooks	the `notebooks` group or extra (JupyterLab); `mesh` for notebook 05	Section 7.3
the tuning tools	a clone; nothing else	Section 8
the mesh pipeline, steps 3–5	the `mesh` group; the Instant Meshes binary	Section 10
the mesh pipeline, steps 1–2	a Python 3.12 environment with CGAL; CoACD and manifold3d; PaMO's container	Section 10
the documents	TeX Live and poppler	Section 11

2 Prerequisites

2.1 The machine

Table 2: What the machine needs, and what it was here.

	needed	here
system	Linux on x86-64 with glibc 2.28 or newer: the wheels of Warp, MuJoCo, NumPy and ImGui are `manylinux_2_28` or `2_27`, and the lock file takes Warp from an x86-64 wheel	Ubuntu 20.04, glibc 2.31, 32 cores, 62 GiB of memory
GPU	an NVIDIA GPU that CUDA supports; Warp's own minimum is a GeForce GTX 9xx. Every number in these documents is from one RTX 4080	RTX 4080, 16 GiB, sm_89
driver	one that supports CUDA 13.0 (the 580 series or newer) for the uv installation; CUDA 12.0 (525 or newer) for pip's (Section 2.2)	580.126.09, CUDA 13.0
CUDA toolkit	none: Warp carries its own compiler (NVRTC)	–
GPU memory	little for a first simulation of 64 worlds; a large batch at a scene's default capacities can ask for more than the card has (Section 14)	16 GiB
disk	0.8 GiB for the clone's files, and its history; 1.8–2.7 GiB for the environment, 2.6 GiB of uv cache, 0.1–4 GiB of kernel cache; 10 GiB more for PaMO's image	423 GiB free

2.2 The driver

```
nvidia-smi
```

Its header names the driver and the newest CUDA version the driver supports:

```
| NVIDIA-SMI 580.126.09             Driver Version: 580.126.09     CUDA Version: 13.0     |
```

- **uv** installs Warp 1.16.0's CUDA 13.0 build (`warp-lang 1.16.0+cu13`). It needs "CUDA Version" 13.0 or higher.
- **pip** installs the build on PyPI, made with CUDA 12.9 (`warp-lang 1.16.0`). It needs 12.0 or higher: Warp relies on CUDA's minor-version compatibility, so any 12.x driver serves.

A driver too old for the build leaves Warp without the GPU: it warns "Insufficient CUDA driver version" and lists only the CPU (Section 14). No CUDA toolkit is needed for either build.

2.3 Software

Table 3: The software each route needs. "Here" is what the commands in this guide ran with.

tool	needed for	here
git	the clone, and pip's git URLs. No Git LFS: the assets are plain git objects	git 2.50
uv	the recommended installation	uv 0.12.5
Python 3.13 or newer	the pip installation only: uv fetches its own Python 3.14	CPython 3.13.5 for pip, uv's 3.14.7
Python 3.12	step 1 of the mesh pipeline (CGAL), and Newton's sims (Section 9); uv fetches it	CPython 3.12.14 and 3.12.12, uv's
singularity or apptainer	step 2 of the mesh pipeline (PaMO's container)	singularity-ce 3.9.5
TeX Live and poppler	building the documents	TeX Live 2019, poppler 0.86

uv, if it is not there yet, installs into `~/.local/bin`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version
```

3 Getting the code

```
git clone https://github.com/martius-lab/lodos
cd lodos
git log -1 --oneline      # the commit you have
du -sh viewer/assets      # about 740M
```

- A copy on the same machine clones with `git clone file:///path/to/lodos`.
- A checkout's files are 761 MiB, of which `viewer/assets/` is 736 MiB (771 MB) in 560 files: the NIST parts 434 MiB, the nut-bolt 102 MiB, the surfaces 81 MiB, the SHARPA hand 65 MiB, the Shadow hand and its Kong toy 53 MiB. The history in `.git` comes on top.
- The assets are ordinary git objects, so a clone has every built-in scene's geometry. There is no Git LFS and no download step.

- Git ignores what the tools write into the clone:
  - the environments, `.venv/` (and `viewer/newton/.venv/`, Section 9);
  - derived meshes in `.scratch/`, and the tuning tools' output in `results/`;
  - `MUJOCO_LOG.TXT` (MuJoCo's warnings) and `imgui.ini` (the viewer's panel layout);
  - the documents' build, `docs/manual/build/`;
  - the mesh pipeline's `.instant-meshes` binary, CGAL's `.pylibs/`, PaMO's `.pamo.sif` (about 10 GiB) and the pipeline's `.tmp/` directory, all in `tools/mesh_processing/`.

4 The recommended installation: uv

4.1 One command

```
uv sync --locked
```

From the clone's root, this does five things:

1. It reads `.python-version` (3.14) and downloads CPython 3.14.7 (34 MiB), unless a Python 3.14 is already there.
2. It creates `.venv/` in the clone.
3. It installs the 70 packages that `uv.lock` pins for the default groups, `vis` and `mesh`: Warp from NVIDIA's CUDA 13 wheel on GitHub, PyTorch's CPU build from PyTorch's own index, everything else from PyPI.
4. It builds Lodos with hatchling and installs it editable: `site-packages/_lodos.pth` puts the clone's root on `sys.path`, so its `lodos/` and `viewer/` are imported in place, and an edit takes effect without reinstalling.
5. It installs the `lodos-viewer` command into `.venv/bin`.

`--locked` stops with an error when `uv.lock` does not match `pyproject.toml`, rather than resolving again. Measured on 2026-10-27: 15 s with an empty cache, which then held 2.6 GiB. `.venv/` is 2.1 GiB, most of it PyTorch (711 MiB), VTK (564 MiB, for PyVista) and Warp (342 MiB). On the same filesystem the environment's files are hard links into uv's cache, so the two do not take twice the space.

Check that Lodos imports from the clone, with the pinned Warp and MuJoCo:

```
.venv/bin/python -c "import lodos, viewer, warp, mujoco; print(lodos.__file__, warp.__version__,
    mujoco.__version__)"
```

It prints `.../lodos/lodos/__init__.py 1.16.0 3.14.0`: the clone's own `lodos/`, Warp 1.16.0 and MuJoCo 3.14.0.

4.2 The dependency groups

Table 4: `pyproject.toml`'s dependency groups, which uv installs. pip has extras of the same names (Section 6).

group	packages	default
`vis`	newton 1.5.1, imgui-bundle, pyglet, matplotlib, mediapy, pillow	yes
`mesh`	torch (the CPU build), trimesh, tqdm, pyvista	yes
`notebooks`	jupyterlab	no

```
uv sync --locked --all-groups       # adds JupyterLab: 61 more packages, 2.2 GiB in all
```

```
uv sync --locked --no-group mesh    # no torch or pyvista: 0.8 GiB; making surfaces needs them
```

The simulator needs none of the groups: with `--no-default-groups`, a built-in scene builds and steps. The viewer needs `vis`; the notebooks `vis` and `notebooks`, and notebook 05, which makes a surface, `mesh` too.

> **uv sync is exact**. `uv sync` makes the environment exactly what its command line asks for. A plain `uv sync` after `--all-groups` uninstalls JupyterLab, and it uninstalls whatever was added by hand with `uv pip install`, such as the mesh tools of Section 10. Give it the same groups every time, or add `--inexact` to keep what is there. `uv run` syncs inexactly.

4.3 Using the environment

```
source .venv/bin/activate          # python, lodos-viewer and jupyter on the PATH
uv run python my_script.py         # or run inside the environment without activating it
.venv/bin/python my_script.py      # or name its interpreter
```

The rest of this guide writes `.venv/bin/...`, so that each command works without activation.

5 What is installed, and from where

Table 5: The components of an installation. Versions are those of `uv.lock`.

component	version	from
Python	3.14.7	uv's own build, as `.python-version` asks; `pyproject.toml` allows 3.13 and newer
Warp (`warp-lang`)	1.16.0+cu13	NVIDIA's GitHub release (`pyproject.toml`'s `tool.uv.sources`); pip takes 1.16.0 from PyPI, built with CUDA 12.9
MuJoCo (`mujoco`)	3.14.0	PyPI: the release that matches MuJoCo Warp 3.14.0, from which Lodos's collision and solver code are adapted
MuJoCo Warp	–	not a dependency: the parts Lodos uses are its own code, adapted from MuJoCo Warp 3.14.0 (`lodos.col.prim`, `lodos.cd.solver`, `lodos.mjcf`)
Newton (`newton`)	1.5.1	PyPI: the viewers' OpenGL window, nothing else. Newton's sims run on Newton 1.7.0, in an environment of their own (Section 9)
PyTorch (`torch`)	2.14.1+cpu	PyTorch's CPU index: the mesh tools' vertex ordering
NumPy, SciPy	2.5.4, 1.18.1	PyPI
the assets	–	`viewer/assets/`, in a clone and in a wheel

5.1 Assets and caches

6 Installing with pip

pip does not read `uv.lock`, so it resolves the newest versions `pyproject.toml` allows, and it does not read `tool.uv.sources`, so Warp and PyTorch come from PyPI. There are three routes.

6.1 Editable, from a clone

```
python3.13 -m venv ~/venvs/lodos         # any Python 3.13 or newer, outside the clone
source ~/venvs/lodos/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cpu    # first, for the mesh extra
```

Table 6: Where an installation reads and writes, and the environment variables that move it.

what	where	moved by
the assets, in a clone or an editable install	`viewer/assets/`	`LODOS_ASSETS`; for one family's, `LODOS_SHARPA_DIR`, `LODOS_NUTBOLT_DIR`, `LODOS_NIST_DIR`, `LODOS_FRANKA_DIR`, `LODOS_SHADOW_KONG_DIR`
the assets, in a wheel	`site-packages/viewer/assets`	the same
derived files (the OBJs MuJoCo takes inertia from)	`.scratch/` in a clone; `~/.cache/lodos` for a wheel	`LODOS_CACHE`; `XDG_CACHE_HOME`
Warp's compiled kernels	`~/.cache/warp/1.16.0`	`WARP_CACHE_PATH`, to which Warp adds the version
uv's downloads and builds	`~/.cache/uv`	`UV_CACHE_DIR`

```
pip install -e ".[vis,mesh,notebooks]"
python -c "import lodos; print(lodos.__file__)"
```

This is the developer's route without uv: 51 s (PyTorch 13 s, the rest 38 s) and 2.7 GiB, on Python 3.13.5. The environment lives outside the clone because `.gitignore` ignores only `.venv/`. The extras are the groups of Table 4; `notebooks` also brings matplotlib and mediapy, which uv's group takes from `vis`.

> **PyTorch from PyPI is the CUDA build.** `pip install ".[mesh]"` alone takes torch from PyPI, whose Linux wheel is built for CUDA: a dry run on 2026-10-27 listed torch 2.14.1 with 19 CUDA, NVIDIA and Triton packages beside it (`cuda-toolkit` 13.0, cuDNN, NCCL and more). Lodos uses torch only to order vertices, on the CPU. Install the CPU build first, as above; pip then keeps it.

6.2 A regular install, from a clone

```
pip install ".[vis]"
```

This builds a wheel of 551 MiB (`lodos`, and `viewer` with its scenes and `viewer/assets/`) and installs it: 39 s, and 1.8 GiB with the `vis` extra. Nothing then refers to the clone. Run Python from outside the clone: inside it, `import lodos` finds the clone's `lodos/` first, since the current directory is on `sys.path`. That works, but it is not what was installed.

6.3 From a git URL, without a clone

```
pip install "lodos[vis] @ git+https://github.com/martius-lab/lodos"           # default branch
pip install "lodos[vis] @ git+https://github.com/martius-lab/lodos@<commit>"  # one commit
```

pip makes a partial clone (`--filter=blob:none`: the history, and only the files of the commit it installs), builds the same wheel and installs it, as `pip install .` does, after the clone. git must be installed. The result is the library, the viewer with its scenes and assets, and `lodos-viewer`. The notebooks, the benchmarks, the tuning tools, the mesh pipeline and the documents are not in the wheel; they need a clone.

Table 7: The four routes, as measured here on 2026-11-04 (uv's time on 2026-10-27). "Time" is with empty caches on a fast network.

	`uv sync --locked`	`pip install -e`	`pip install .`	pip, git URL
versions	locked	newest allowed	newest allowed	newest allowed
Warp build	CUDA 13.0	CUDA 12.9	CUDA 12.9	CUDA 12.9
`lodos`, `viewer`	the clone's, in place	the clone's, in place	copied	copied
assets	the clone's	the clone's	in the wheel	in the wheel
notebooks, tools	in the clone	in the clone	in the clone	none
time	15 s	51 s	39 s	39 s and the clone
size	2.1 GiB (2.2 with notebooks)	2.7 GiB, `vis`, `mesh`, `notebooks`	1.8 GiB, `vis`	1.8 GiB, `vis`

6.4 What each route gives

pip's resolution differed from the lock in 26 of its 134 packages, among them torch 2.15.0 for the lock's 2.14.1, imgui-bundle 1.93.0 for 1.92.900 and trimesh 5.2.0 for 5.1.1. Under pip, on the CUDA 12.9 build, the imports and the first simulation of Section 7.2 ran as under uv (the lowest armadillo at 0.357 m, nothing overflowed); the notebooks were run under uv only.

To give pip the same Warp build as uv, on a driver that supports CUDA 13.0, install NVIDIA's wheel after the rest. It satisfies `warp-lang==1.16.0`, and `pip check` finds nothing broken:

```
W=https://github.com/NVIDIA/warp/releases/download/v1.16.0
pip install "warp-lang @ $W/warp_lang-1.16.0+cu13-py3-none-manylinux_2_28_x86_64.whl"
```

7 Verifying the installation

7.1 Warp and the GPU

```
.venv/bin/python -c "import warp as wp; wp.init()"
```

```
Warp 1.16.0 initialized:
   CUDA Toolkit 13.0, Driver 13.0
   Devices:
     "cpu"      : "x86_64"
     "cuda:0"   : "NVIDIA GeForce RTX 4080" (16 GiB, sm_89, mempool enabled)
   Kernel cache:
     /home/<user>/.cache/warp/1.16.0
```

- "CUDA Toolkit" is the Warp build: 13.0 under uv, 12.9 under pip. "Driver" is the newest CUDA the driver supports.
- There must be a `cuda:0` line. Without it, Lodos has no GPU to run on (Section 14).

7.2 A first simulation

```
import warp as wp
from lodos import step
from viewer.scenes import armadillo

# MJWarp's capacities must hold Lodos's contacts too: at MJWarp's defaults it can drop them.
# These are the viewer's for five armadillos as surfaces.
sim = armadillo.build(batch=64, seed=0, nconmax=4096, njmax=12288)
replay = step.make_step_graph(sim)   # the whole step, captured as one CUDA graph
for _ in range(2000):                # four seconds at dt = 2 ms
    replay()
wp.synchronize()
z = sim.drbd.qpos.numpy()[:, 2::7]   # every armadillo's height, in every world
print(f"lowest {z.min():.3f} m, highest {z.max():.3f} m")
```

Saved as `first.py` and run with `.venv/bin/python first.py`, it drops five armadillos in each of 64 worlds and lets them settle for four seconds, under the solver's default, exact mode (`solver_options=dict(mode="mj")` runs MuJoCo's). On 2026-11-04, on the RTX 4080, it printed

```
lowest 0.357 m, highest 1.368 m
```

and on a second run a highest of 1.897 m. Two more runs, which read the counts every step, overflowed nothing: at most 782 contacts of the 262 144 the batch holds, and 57 rows a world. An armadillo lying on the floor has its origin 0.357 m up; the highest lies on the pile. The drop is chaotic, so the highest differs from run to run, and with the GPU and the code. A `nan`, or a lowest height near zero (a body through the floor), is a problem.

The first run compiles. Warp compiles each kernel the first time it is launched, and keeps it in its kernel cache, so on an empty cache the first build and capture waits for the compiler. With the cache warm, building and capturing the scene took 0.4 s (2026-11-04). The cache grows with every scene and option run.

7.3 The notebooks

```
uv sync --locked --all-groups          # once: JupyterLab
.venv/bin/jupyter lab notebooks/
```

Open `01_quickstart.ipynb` and run every cell. Without a browser, the same notebook runs from the command line:

```
.venv/bin/jupyter nbconvert --to notebook --execute --output-dir /tmp/nb \
    notebooks/01_quickstart.ipynb
```

Its first output is `device: cuda:0`; its last says how fast the armadillos still move after 8 s. The notebooks draw with MuJoCo's renderer, and set `MUJOCO_GL=egl` when it is unset, so they render without a display. Their stored outputs are from an RTX 4080. `notebooks/README.md` lists all five. The fifth, `05_make_a_surface`, makes a Lodos surface from a triangle mesh and needs the `mesh` group, a default one; under pip, the extras are `".[vis,notebooks,mesh]"`.

7.4 The live viewer

```
.venv/bin/lodos-viewer arm_5_sdf      # the same as: .venv/bin/python -m viewer arm_5_sdf
.venv/bin/lodos-viewer --help
```

- A window titled "Lodos live", its sidebar headed "Newton Viewer", opens at 85 % of the screen with the five-armadillo drop loaded, at frame 0 under Lodos. *Play*, in the timeline at the bottom, runs it.
- Without a scene's name the window opens empty, and a scene is loaded from its Scene panel. `--sim` picks the sim it starts under: `lodos`, the default, or, once Newton's environment is installed (Section 9), `mjwarp` or `kamino`. `--tuned` applies the knobs of each scene's tuning file.
- It needs an OpenGL display (Section 12). Without one it stops with this error:

```
pyglet.display.xlib.NoSuchDisplayException: Cannot connect to "None"
```

On the code as of 2026-11-04 the window was not run for this guide; `lodos-viewer --help` was. *The Live Viewer* (`docs/visualizer.pdf`) is the viewer's manual.

8 The tuning tools

`tools/performance_tuning/` is a directory of scripts that run in the main environment from a clone. There is nothing to install, and a wheel does not have them.

```
.venv/bin/python tools/performance_tuning/tune.py --help
.venv/bin/python tools/performance_tuning/profile_scene.py arm_5_sdf --B 1024 \
    --tuning tools/performance_tuning/tuned/arm_5_sdf.tuning.json
```

The second command times each stage of one step of the five-armadillo drop; it was not run for this guide. It warns that the tuning file was measured at 4 096 worlds, the file's `B`, which is expected at 1 024. The tools time on the GPU, so run them with nothing else on it. Their window is the viewer's Performance panel. *Performance Tuning* (`docs/performance_tuning.pdf`) is their guide.

> **tune.py writes over the shipped tuning files**. The tools write their state sets and reports into `results/`, which git ignores. `tune.py` writes its tuning file and its report into `tools/performance_tuning/tuned/`, the directory of the files that ship, unless `--out-dir` says otherwise. Give it `--out-dir results/tuning`, as the guides' examples do, unless the point is to replace a shipped file.

9 Newton's sims, for comparison

The viewer's Newton side, `viewer/newton/`, runs Newton's own solvers on the built-in scenes: Newton 1.7.0's `SolverMuJoCo` on upstream MuJoCo Warp 3.15, and its `SolverKamino`. They need an environment of their own, because Newton 1.7.0 runs on MuJoCo 3.15 (its lock: MuJoCo 3.15.0, MuJoCo Warp 3.15.0, Warp 1.18.0), where Lodos pins MuJoCo 3.14.0 and Warp 1.16.0. It is a uv project of its own, off by default, and nothing in it imports Lodos:

```
cd viewer/newton && uv sync --locked     # Python 3.12, newton[sim]==1.7.0, trimesh, scipy: 0.7
    GiB
```

Once it exists, the viewer's sim menu offers MJWarp and Kamino beside Lodos, and `python -m viewer SCENE --sim mjwarp` (or `kamino`) starts on one; the viewer runs them in a process of their own, in this environment. `LODOS_NEWTON_PYTHON`, set in the environment, points the viewer at another interpreter with Newton installed. `viewer/newton/README.md` says what the import of a scene changes. The environment was not installed afresh for this guide on 2026-11-04: its lock checked as current (`uv sync --locked --dry-run`), and 0.7 GiB is the size of an existing one.

10 The mesh pipeline's tools

`tools/mesh_processing/` makes new collision geometry, in five steps from an MJCF's meshes to convex hulls and Lodos surfaces. *Creating a Simulation from an MJCF* (`docs/creating_new_sim_from_mjcf.pdf`) explains the steps, `tools/mesh_processing/README.md` how to run them, and each script's docstring its options. Nothing in it is needed to simulate. Steps 3–5 run in the main environment once Instant Meshes is in place; steps 1 and 2 need programs that are not in it (Table 8).

Table 8: What each step of the mesh pipeline needs beyond the main environment, and what of it was checked here.

step	script	needs	checked here
1	`fuse_meshes_by_body.py`	Python 3.12 with CGAL, scikit-image, Warp, MuJoCo and trimesh	ran on a two-body model, without its window
2	`calibrate_cp_decomp.py`	PaMO's container (singularity); CoACD; manifold3d	CoACD and manifold3d on a test mesh; the container was not pulled
3	`interactive_remesh.py`	the Instant Meshes binary	downloaded and compared; not run
4	`fix_nonmanifold_meshes.py`	the same	the same
5	`create_surface_data.py`	the GPU	not run

10.1 Instant Meshes, for steps 3 and 4

```
curl -L -o /tmp/instant-meshes-linux.zip \
    https://instant-meshes.s3.eu-central-1.amazonaws.com/instant-meshes-linux.zip
unzip -p /tmp/instant-meshes-linux.zip "Instant Meshes" > tools/mesh_processing/.instant-meshes
chmod +x tools/mesh_processing/.instant-meshes
```

- The link is the Linux build that Instant Meshes' README points to; there is no package. The zip holds one file, `Instant Meshes`, 2.6 MiB, built on 2019-11-03. Its SHA-256 begins `a1946606`: the same file as the copy the pipeline's runs in these documents used.
- The pipeline runs it as `tools/mesh_processing/.instant-meshes`, in batch mode, without a window. It links X11's client libraries all the same (libX11, libXrandr, libXinerama, libXcursor, libXxf86vm), which a minimal server may lack: on Ubuntu, `sudo apt install libx11-6 libxrandr2 libxinerama1 libxcursor1 libxxf86vm1`.
- The binary was not run for this guide; *Creating a Simulation from an MJCF* reports the runs of steps 3 and 4 with it.

10.2 CoACD, manifold3d and scikit-image, for steps 1 and 2

```
uv pip install scikit-image coacd manifold3d
```

- They install into the main environment on Python 3.14: CoACD 1.0.14, manifold3d 3.5.4 and scikit-image 0.26.0. Here they decomposed and joined a test mesh.
- No extra installs them: `pyproject.toml` names none of them.
- A later `uv sync` removes them (Section 4.2). Add `--inexact` to it, or let `uv run` bring them each time, which was checked here too:

```
uv run --with coacd --with manifold3d --with scikit-image \
```

```
python tools/mesh_processing/run_pipeline.py path/to/robot.xml --from 2
```

10.3 PaMO, for step 2

```
singularity pull tools/mesh_processing/.pamo.sif docker://sarahwei0210/pamo:0.0.2
```

- PaMO decimates each body before CoACD decomposes it. It needs Python 3.10, NumPy 1.26 and a Warp of its own, so it runs from its published container, one `singularity exec --nv` per body, on the GPU.
- The image is 10.5 GB (9.8 GiB) compressed on Docker Hub. The pull was not run for this guide.
- `pamo_driver.py` calls the command `singularity`. Apptainer, Singularity's successor, provides a command of that name.
- PaMO is AGPL-3.0: only its output meshes are kept, and none of its code is linked.

10.4 CGAL and step 1: a Python 3.12 environment

CGAL's Python bindings, the PyPI package `cgal` 6.0.1, have wheels for CPython 3.8 to 3.12 only. Step 1 therefore runs from an environment of its own:

```
uv venv --python 3.12 ~/venvs/lodos-step1
uv pip install --python ~/venvs/lodos-step1 cgal "warp-lang==1.16.0" "mujoco==3.14.0" \
    trimesh scikit-image "newton==1.5.1" imgui-bundle "pyglet<3"
~/venvs/lodos-step1/bin/python tools/mesh_processing/fuse_meshes_by_body.py path/to/robot.xml
.venv/bin/python tools/mesh_processing/run_pipeline.py path/to/robot.xml --from 2
```

- uv fetched Python 3.12.14 and installed the rest in 4 s; the environment is 0.9 GiB.
- `newton`, `imgui-bundle` and `pyglet` are for step 1's window. With `--no-view`, step 1 needs none of them.
- `run_pipeline.py` runs every step with the interpreter that runs it, so from the main environment it starts at step 2.
- Checked on 2026-10-27, on a model of two closed meshes: step 1 wrapped both into watertight solids in 12 s, without its window and without the GPU. Its winding-number pass, the part that uses the GPU, runs only on bodies that are not already closed.

> **Never let Python 3.14 load CGAL.** `fuse_meshes_by_body.py`'s docstring installs CGAL with `pip install --no-deps --target` into `tools/mesh_processing/.pylibs`, a directory where step 1 looks when CGAL is not importable. From the 3.12 environment that works. From the main environment it does not: Python 3.14 imports those 3.12 binaries without complaint, since their file names carry no version, and then crashes with a segmentation fault in the alpha wrap. Without `.pylibs`, the same run stops cleanly with `ModuleNotFoundError: No module named 'CGAL'`. Install CGAL into the 3.12 environment, as above, and leave `.pylibs` absent.

10.5 The windows

Every step opens a Newton viewer on its result, and steps 2 and 3 wait in theirs for choices. `--no-view`, on each script and on `run_pipeline.py`, skips the windows, and `run_pipeline.py --replay DIR` runs steps 2 and 3 from the choices a previous run recorded. A machine without a display uses those (Section 12).

11 The documents' toolchain

The documents are LaTeX sources in `docs/manual/`, which `docs/manual/build.sh` builds into `docs/<name>.pdf` and `docs/<name>.md`. It needs `pdflatex` with the packages `docs/manual/preamble.tex` loads, poppler's `pdftoppm` (the figures' PNGs) and `pdfinfo` (the page count), and the main environment's Python (matplotlib, for the data figures). On Ubuntu:

```
sudo apt install texlive-latex-extra texlive-pictures texlive-science \
    texlive-fonts-recommended texlive-fonts-extra poppler-utils
```

This line was not run for this guide: the machine already had TeX Live's base, LaTeX, recommended, extra and pictures packages, and poppler, and the rest in a user tree (below). Table 9 says where the less common packages come from.

This prints nothing when every package is found:

```
for p in libertine newtxmath zi4 tcolorbox titlesec enumitem needspace cleveref \
         algorithm algpseudocode pgfplots standalone; do
    kpsewhich $p.sty > /dev/null || echo "missing: $p"
done
```

Table 9: Where the preamble's less common packages come from on Ubuntu. Those marked * are not installed system-wide on this machine but in its user tree, so their package names are Debian's, not checked here.

LaTeX packages	Ubuntu package
tcolorbox, titlesec, enumitem, needspace, cleveref, standalone	`texlive-latex-extra`
microtype, caption, booktabs, listings	`texlive-latex-recommended`
tikz, pgfplots	`texlive-pictures`
algorithm, algpseudocode *	`texlive-science`
libertine, newtxmath, zi4 (Inconsolata) *	`texlive-fonts-extra`
the URW fonts behind helvet *	`texlive-fonts-recommended`

Without root. TeX searches the user tree `~/texmf` (`kpsewhich -var-value TEXMFHOME`) before the system's, so packages from CTAN can be unpacked there. This machine has libertine, newtx, inconsolata, algorithms, algorithmicx and the URW fonts (helvetic, times, courier, symbol, zapfding) there, their font maps enabled with `updmap-user`. That was done before this guide and not repeated for it.

Building. From the clone's root, one document or all of them:

```
docs/manual/build.sh installation_guide     # one document
docs/manual/build.sh                        # all of them
```

- Each document takes three `pdflatex` passes, for the references, and then `tex2md.py` for its Markdown; the script prints the page count. In the fresh clone of 2026-10-27, *Performance Tuning* built in 3.5 s.
- It runs `.venv/bin/python`; `PYTHON=/path/to/python docs/manual/build.sh` runs another.
- In a fresh clone the first build also re-renders the seven data figures, because git gives `make_figures.py` a later timestamp than their PDFs. `make_figures.py` needs no GPU, and the PDFs change only in their metadata: `git checkout docs/manual/fig` discards them. It draws in Linux Libertine when it finds the font in `~/texmf`, and in DejaVu Sans otherwise.
- `make_data.py`, which measures the data behind those figures, needs the GPU and is not part of a build. `make_tables.py` rewrites the guides' tables from `tools/performance_tuning/tuned/`, with no GPU.

12 Machines without a display

Everything but the windows. None of it needs a display. Every run in this guide but the viewer's had none (an empty `DISPLAY`): the first simulation, the notebook (MuJoCo renders through EGL, which comes with the NVIDIA driver), the tuning tools and step 1 of the mesh pipeline with `--no-view`. The mesh pipeline's other steps run without windows under `--no-view` and `--replay` (Section 10.5).

The live viewer. It needs an X display with OpenGL 3.3 or newer. On a desktop that is the session's own. On a machine without one, Xvfb, a virtual X server, will do:

```
xvfb-run -a -s "-screen 0 1920x1080x24" .venv/bin/lodos-viewer arm_5_sdf
```

- On Ubuntu it needs the packages `xvfb` and `libgl1-mesa-dri` (here Xvfb 1.20.13 and Mesa 21.2.6). Mesa then renders the window on the CPU (llvmpipe, OpenGL 4.5), while the simulation stays on the GPU.
- Measured on 2026-10-27: 43 frames a second with the five-armadillo drop loaded, in a window of 1632 by 918 pixels.
- It prints `Warp CUDA error 999: unknown error (in function wp_cuda_graphics_register_gl_buffer ...)` once. Newton's frame grab cannot share a Mesa buffer with CUDA, and falls back; nothing else is affected.

- To see and drive a virtual display, attach a VNC server to it, or use a remote desktop whose X server runs on the machine's NVIDIA GPU. Neither was tried for this guide; nor was X forwarding over ssh.

Containers. A container must be given the GPU: `docker run --gpus all`, or `singularity exec --nv`. Without it, Warp reports `no CUDA-capable device is detected` (Section 14).

13 Keeping an installation current

```
git pull
uv sync --locked --all-groups       # the same groups as the first time
```

- When `uv.lock` changed, this installs exactly the new lock; when it did not, it changes nothing (under 0.1 s here).
- `--locked` fails with "The lockfile at `uv.lock` needs to be updated" when `pyproject.toml` was changed without `uv lock`. Changing a dependency means editing `pyproject.toml`, running `uv lock` and committing both (*Development Guide*).
- Under pip, run the same `pip install -e ".[...]"` again after a pull that changed `pyproject.toml`.
- A new Warp version compiles into a new directory of the kernel cache. The old ones can be deleted: this machine's `~/.cache/warp` holds seven versions, 1.16.0's alone 4.0 GiB.

14 Troubleshooting

Table 10: Symptoms, causes and fixes. The messages were seen here, except those of PaMO, Instant Meshes and LaTeX, which are quoted from the code and from LaTeX's error format.

symptom	cause	fix
"Insufficient CUDA driver version"; no `cuda:0` among Warp's devices	the driver is older than the Warp build: uv's needs CUDA 13.0, PyPI's 12.0	a newer driver (the 580 series for uv); or the pip route, which takes the CUDA 12.9 build (Section 2.2 and Section 6)
`Warp CUDA error 100: no CUDA-capable device is detected`	no GPU visible: `CUDA_VISIBLE_DEVICES` empty, no driver, or a container without access to the GPU	`nvidia-smi`; in a container, `docker run --gpus all` or `singularity exec --nv`
the first run waits before anything moves	Warp compiling kernels into its cache	nothing: each kernel compiles once (Section 7.2)
kernels that fail to load	a damaged kernel cache: an interrupted compile, a full disk	`python -c "import warp; warp.clear_kernel_cache()"`, or delete `~/.cache/warp/1.16.0`; it recompiles
`RuntimeError: Failed to allocate 6442450944 bytes on device 'cuda:0'`	out of GPU memory. A scene's own capacities are sized for the viewer's one world: the five-armadillo drop at 4 096 worlds asks for 6 GiB for the constraint Jacobian alone	fewer worlds; the scene's tuning file (`tools/performance_tuning/tuned/`), whose capacities are sized to the scene's peaks; other processes on the GPU (`nvidia-smi`)

symptom	cause	fix
`pyglet.display.xlib.NoSuchDisplayException: Cannot connect to "None"`	the viewer started without an X display: over ssh, on a server, in a batch job	a desktop session, or Xvfb (Section 12)
`Warp CUDA error 999` in `wp_cuda_graphics_register_gl_buffer`, under Xvfb	Newton's frame grab cannot share Mesa's buffers with CUDA	nothing: the viewer runs on
`ModuleNotFoundError: No module named 'torch'` on importing `lodos.col.sdf.io`	the `mesh` group or extra is missing	`uv sync --locked`, whose default groups include it; or `pip install -e ".[mesh]"`
JupyterLab, or the mesh tools, gone after a `uv sync`	`uv sync` is exact	the same groups every time, or `--inexact` (Section 4.2)
pip downloads CUDA and NVIDIA packages for torch	PyPI's torch is the CUDA build	the CPU build first (Section 6.1)
`error: The lockfile at uv.lock needs to be updated, but --locked was provided.`	`pyproject.toml` changed without `uv lock`	`uv lock`, and commit both files
`Segmentation fault` in step 1 of the mesh pipeline	Python 3.14 running the 3.12 CGAL in `tools/mesh_processing/.pylibs`	step 1 from the 3.12 environment; delete `.pylibs` (Section 10.4)
`ModuleNotFoundError: No module named 'CGAL'`	step 1 run from the main environment	step 1 from the 3.12 environment (Section 10.4)
`PaMO container not found`	the image is not pulled	`singularity pull` (Section 10.3)
`FileNotFoundError` or `PermissionError` naming `tools/mesh_processing/.instant-meshes`	the binary is not in place, or not executable	Section 10.1
a LaTeX error that a file such as `libertine.sty` was not found	a LaTeX package is missing	Section 11
`git status` lists `docs/manual/fig/*.pdf` after a build	the first build in a clone re-renders the data figures	`git checkout docs/manual/fig`